

Lab 3: AVR I/O and Software Time Delay

Microprocessors (010153521) | Semester 1/2026

Duration: 3 hours (in-lab) | **Topic:** Bare-metal AVR assembly in Microchip Studio, hex upload to an Arduino Uno R3 (ATmega328P) via AVRdude, and oscilloscope-verified software time delays

Objectives

- Configure a real I/O pin as an output with DDRB and drive it with OUT – direct I/O addressing on physical hardware, not a simulator (Lecture 2, §2.2-2.3).
- Write a correct, buildable AVR assembly program in Microchip Studio: correct program structure, .INCLUDE, .ORG, and an initialized stack pointer before any CALL/RET (Lecture 2, §2.5-2.6; Lecture 3, §3.2).
- Build single- and nested-counter delay loops with DEC/BRNE, wrapped in a CALL/RET subroutine (Lecture 3, §3.1-3.2).
- Calculate, by hand, the number of loop iterations needed for a target time delay at the Arduino Uno’s actual crystal frequency (16 MHz), using the cycle-counting method of Lecture 3, §3.3.
- Assemble the program to an Intel HEX file and upload it to an Arduino Uno R3 using avrdude’s Arduino (bootloader) programmer – entirely from the command line, without the Arduino IDE.
- Verify the real output waveform on an oscilloscope – period, frequency, duty cycle – against your hand calculation, and account for the difference.

Quick Reference: I/O on This Board

The ATmega328P’s data memory, SFR names, and IN/OUT mechanics are exactly as in Lecture 2 – only the chip and the pin names change.

Port	Dir. reg.	Arduino Uno pins
PORTB (PB0-PB5)	DDRB	D8-D13 (PB5 = D13, on-board LED)
PORTC (PC0-PC5)	DDRC	A0-A5
PORTD (PD0-PD7)	DDRD	D0-D7 (PD0/PD1 = USB RXD/TXD – do not drive)

DDRx bit = 1 makes that pin an *output*; bit = 0 leaves it an *input* (0 is the reset default, i.e. *every* pin starts as an input). OUT PORTx,Rr only *drives* a pin once its DDRx bit is set – otherwise it just toggles that pin’s internal pull-up (see the “Find the Bug” exercise). Register names such as PB5 and DDRB come from m328Pdef.inc, exactly as M32DEF.INC supplied PORTB in Lecture 2’s Program 2-1.

Quick Reference: Delay Math at 16 MHz

The Arduino Uno’s crystal is **16 MHz**, so $T_{\text{cyc}} = 1/16\text{MHz} = 62.5\text{ ns}$ – half the cycle time used in Lecture 3’s worked examples (mostly 1 MHz or 8 MHz). The delay formula is unchanged:

$$\text{delay} = [1 + (C \times N) - 1 + 4] \times T_{\text{cyc}}$$

where N is the loop count (counter register), C is the cycles used per pass *including* the branch when it is taken, the leading 1 accounts for loading the counter, the -1 corrects for the final (non-taken) branch, and the 4 accounts for RET.

Worked demo (a different target from any lab exercise) – aim for 100 μs : target cycles = $100,000\text{ns}/62.5\text{ns} = 1600$. Choose $N = 200$ and pad the loop body with 5 NOPs, so $C = 5(\text{NOP}) + 1(\text{DEC}) + 2(\text{BRNE, taken}) = 8$ cycles/pass.

$$[1 + (8 \times 200) - 1 + 4] \times 62.5\text{ns} = 1604 \times 62.5\text{ns} = 100.25\text{ } \mu\text{s} \quad (\text{error} = +0.25\%)$$

Perfect round numbers are rare – getting within a percent or two of the target and then *measuring* the real error on the scope is the point of this lab, not hitting the target exactly (see “Verifying Delays in Practice,” Lecture 3).

Quick Reference: Your Individual Target

Exercises 2 and 3 ask for *your own* target, not a shared one – so a copied `.asm` won’t hit the right numbers. Derive it from your student ID with this hash:

$$D = \text{sum of every digit in your student ID}$$

Exercise	Your target
2 (single-loop delay)	$T_2 = 0.50 \text{ ms} + (D \bmod 20) \times 0.05 \text{ ms}$
3 (nested-loop square wave)	$f_3 = 500 \text{ Hz} + (D \bmod 30) \times 50 \text{ Hz}$

Worked example (a made-up 13-digit ID, same format as yours – do not reuse these numbers): ID 1029384756123 $\rightarrow D = 1+0+2+9+3+8+4+7+5+6+1+2+3 = 51$. Then $T_2 = 0.50 + (51 \bmod 20) \times 0.05 = 0.50 + 11 \times 0.05 = 1.05 \text{ ms}$, and $f_3 = 500 + (51 \bmod 30) \times 50 = 500 + 21 \times 50 = 1550 \text{ Hz}$.

Compute your own D , T_2 , and f_3 now and record them at the top of your lab notebook – your instructor will spot-check that your measured results match *your* numbers. Working in pairs? Use whichever partner’s ID is listed first on your roster.

Toolchain: From Source Code to Oscilloscope Trace

1. Write the `.asm` source in **Microchip Studio** (Assembler project, device ATmega328P).
2. **Build** (F7). Studio’s assembler produces the usual family of output files (§2.7) in the project’s **Debug** folder, including a ready-to-burn `.hex` file and a `.lst` listing.
3. **(Recommended) Simulate** the `.hex` in an online simulator such as **Wokwi** to catch logic mistakes – *not* timing – before touching the real board.
4. Upload the `.hex` file to the board with **avrdude**, run from a terminal – *not* Studio’s built-in Device Programming dialog, which targets ISP/debugWIRE hardware programmers, not the Uno’s onboard USB-serial bootloader.
5. Probe the target pin with an **oscilloscope** and compare the measured waveform against your hand calculation.

Setting Up the Microchip Studio Project

1. **File > New > Project... > Assembler > AVR Assembler**. Name the project (e.g. Lab3_Delay).
2. On the device-selection dialog, choose **ATmega328P** (*not* ATmega328 – the “P” variant is what ships on the Uno R3).
3. Replace the generated `main.asm` with the skeleton in Listing 1 below.

The Skeleton Program

Listing 1: Skeleton: blink-check on PB5 (Arduino D13). The delay is a placeholder – not yet timed.

```
;Lab 3 skeleton -- Arduino Uno R3 (ATmega328P), 16 MHz crystal

.INCLUDE "m328Pdef.inc"

.ORG 0x0000
    LDI    R16, HIGH(RAMEND)    ;initialize the stack pointer --
    OUT    SPH, R16             ;required before any CALL/RCALL/RET
```

```

        LDI    R16, LOW(RAMEND)
        OUT    SPL, R16

        LDI    R16, (1<<PB5)      ;PB5 = output (Arduino D13), all other
        OUT    DDRB, R16          ;PORTB pins left as inputs

MAIN:    LDI    R16, (1<<PB5)      ;drive PB5 high
        OUT    PORTB, R16
        RCALL  DELAY
        LDI    R16, 0x00          ;drive PB5 low
        OUT    PORTB, R16
        RCALL  DELAY
        RJMP   MAIN

;----- delay subroutine (placeholder -- retune this in Exercise 2) -----
DELAY:   LDI    R20, 0xFF
AGAIN:   DEC    R20
        BRNE   AGAIN
        RET

```

* We do *not* reserve interrupt vector slots: no interrupt is ever enabled in this program, so `.ORG 0x0000` places our first instruction directly at the reset vector – exactly like Program 2-1’s `.ORG 00`.

Simulate First (Recommended): Wokwi

Before burning a real chip, it is worth checking your program’s *logic* in a free online simulator such as **Wokwi** (wokwi.com):

1. Start a new Wokwi **Arduino Uno** project.
2. Upload the `.hex` file Microchip Studio just built as the project’s firmware – Wokwi’s Arduino Uno will run any valid AVR `.hex`, whether or not Wokwi itself compiled it. No Arduino sketch is needed.
3. Wire a virtual LED (or Wokwi’s logic analyzer) to the pin under test and press Run.

This catches the mistakes that have nothing to do with timing – a missing `DDRB` write, a typo’d register, a loop that never exits – instantly and for free, with no board plugged in.

What Wokwi cannot tell you: its simulated clock is *not* cycle-accurate to a real 16 MHz crystal, so it cannot confirm whether your delay actually hits your target T_2 or f_3 . Use it only to confirm the waveform’s *shape* – does the pin toggle at all, does the counter pattern look right – never to answer this lab’s timing questions. Only the physical oscilloscope measurement counts for that.

Uploading with AVRdude

`avrdude` talks to the Uno’s factory-installed bootloader over the same USB cable used for programming from the Arduino IDE – the `arduino` programmer ID selects the STK500v1 protocol that bootloader speaks, at 115200 baud.

```
avrdude -c arduino -p atmega328p -P <port> -b 115200 -U flash:w:Lab3_Delay.hex:i
```

Flag	Meaning
-c arduino	Programmer type: Arduino/Optiboot bootloader (STK500v1).
-p atmega328p	Target part.
-P <port>	Serial port: COM5 (Windows), /dev/cu.usbmodemXXXX (macOS), /dev/ttyACM0 (Linux). Some Uno clones use a CH340 USB-serial chip and need a separate driver before the port appears.
-b 115200	Bootloader baud rate (Uno-specific; older boards use 19200).
-U flash:w:file.hex:i	Write file.hex (Intel HEX format) to flash.

Before uploading: close the Arduino IDE’s Serial Monitor and any terminal program holding the port open – `avrdude` needs exclusive access, and a busy port is the most common upload failure (Appendix A).

Never pass `-U lfuse:w:...`, `hfuse`, or `efuse` on this board unless explicitly instructed to. The Uno’s fuses are what make the bootloader run at power-up; writing the wrong fuse value can make the board unreachable by `avrdude` (a full recovery then needs a hardware ISP programmer).

Connecting the Oscilloscope

- Probe tip on the pin under test (e.g. the D13 header pin); probe ground clip on any Uno **GND** pin. A shared ground between board and scope is required for a clean trace.
- Start with the vertical scale at 1-2 V/div (logic swings 0-5 V) and the timebase set so that 2-3 full periods fit on screen; then use the scope’s auto-measure or cursors to read period, frequency, and duty cycle.
- If the trace looks noisy or doesn’t trigger, check that `DDRB` was actually set for that pin (see the “Find the Bug” exercise) before suspecting the scope.

Quick Experiments (Try It)

Try It 1: Hello, pin 13

Build the skeleton exactly as given. If you like, load the `.hex` into Wokwi first and confirm the virtual pin toggles – a quick, board-free sanity check on the *logic*. Then upload the real `.hex` with `avrdude` and probe D13. The placeholder delay is deliberately *not* timed to anything meaningful – you are not trying to see it blink. Confirm only that the scope shows *a* periodic square wave: this is your checkpoint that editor → assembler → `avrdude` → real hardware all works, before any precision math begins.

Try It 2: Read the `.lst` file

Open the project’s `.lst` listing file (Studio writes it next to the `.hex`) and find the machine code generated for the `RCALL DELAY` and `BRNE AGAIN` lines.

1. Is `RCALL` 2 bytes or 4, in this listing? Does that match §3.1’s description of `RCALL` versus `CALL`?
2. Using the addresses shown in the listing (not the source line numbers), recompute `BRNE AGAIN`’s relative displacement `k` by hand, the same way Example 3-7 did. Does it match the machine code you see?

In-Lab Exercises (target: 3 hours)

In-Lab Exercise 1: Toolchain checkpoint (25 min)

Repeat Try It 1 as a graded checkpoint: project set up for ATmega328P, skeleton typed in exactly, built, and uploaded with `avrdude`. Record the exact `avrdude` command you used (including your port name) and a screenshot or sketch of the oscilloscope trace on D13. If `avrdude` fails to connect, work through Appendix A before asking for help.

In-Lab Exercise 2: Your precisely timed interval, T_2 (35 min)

Using the “Your Individual Target” box above, compute your own D and T_2 if you haven’t already. Replace the placeholder `DELAY` subroutine with a *single*-counter loop (counter register + `NOP` padding + `DEC/BRNE`), following the method of the 100 μ s worked demo above, tuned for a delay as close as possible to **your** T_2 at 16 MHz. Show your full cycle accounting (N , number of `NOP`s, C , total cycles, total time, and the % error against T_2) in your lab notebook.

1. Wire it into the skeleton’s `MAIN` toggle loop – with two calls to `DELAY` per period, PB5 should now produce a square wave with a period of $2 \times T_2$ (frequency $1/(2T_2)$).
2. Build, upload, and measure the actual period and frequency on the oscilloscope. Compute the % error between your *hand-calculated* period and the *measured* one, and compare it to the % error you calculated against T_2 above – are they close? If not, what did your hand calculation leave out?

In-Lab Exercise 3: Your square wave with a nested loop, f_3 (55 min)

A single 8-bit counter tops out at 255 passes – not enough range for finer control without absurd `NOP` padding. Using your f_3 from the box above, design a *nested* two-register loop (as in Lecture 3’s Example 3-18) for a half-period as close as possible to $1/(2f_3)$, giving an overall square wave at **your** f_3 , roughly 50% duty-cycle, on PB5.

1. Calculate your two register values (N_1 , N_2) and the resulting delay by hand before touching the keyboard.
2. Build, upload, and measure the frequency on the oscilloscope.
3. Tune your register constants and rebuild until the *measured* frequency is within $\pm 2\%$ of f_3 – a realistic engineering tolerance, not exact equality. Keep a small table in your notebook: attempt #, N_1 , N_2 , calculated period, measured period, % error.

In-Lab Exercise 4: Binary ripple counter on PORTD (35 min)

Combine §2.2-2.3’s `OUT` addressing with §3.1-3.3’s counting and delay techniques: maintain an 8-bit counter register, `INC` it once per step, shift it left *twice* with `LSL` before each `OUT PORTD,Rd`, and hold it for a fixed per-step delay (reuse or rescale your `DELAY` subroutine) before incrementing again.

- The two `LSL`s matter: they guarantee bits PD0/PD1 stay 0 for every count, so the USB-serial lines are never driven while the board stays connected – only PD2 and above carry the counting pattern.
- Probe **PD2** (the fastest-toggling driven bit). Its period should equal exactly **twice** your programmed per-step delay – explain why in terms of how an incrementing binary count flips its LSB.
- Probe **PD3**. Its period should be exactly **twice** PD2’s – verifying, directly on the scope, the binary-counting relationship between adjacent bits.

AI Collaboration**Try these prompts with an AI tool:**

- “I need a nested AVR delay loop as close as possible to my assigned half-period, $1/(2f_3)$, at 16 MHz. Show the cycle-counting, not just a guessed answer.”
- “What does `avrdude: stk500_getsync(): not in sync` mean, and what are the most common causes on an Arduino Uno?”
- “What is the difference between setting a `DDRx` bit and setting the corresponding `PORTx` bit on an AVR pin configured as an input?”

Critical check – verify, don’t just accept: If an AI tool gives you register values for a target delay, do *not* trust the number – redo its cycle count by hand, instruction by instruction, the same way you did for the 100 μ s demo. Then check it a second way: does the *oscilloscope* agree, within a percent or two, with either calculation?

Know the limit: AI tools are good at producing *a* delay loop that is plausible-looking but will happily miscount the cycle cost of the final (non-taken) branch, or forget the `RET` overhead entirely – exactly the kind of off-by-a-few-cycles error that is invisible in the source but visible on a scope trace. The oscilloscope is your ground truth in this lab, not the AI’s arithmetic.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Nested-loop delay at 8 MHz

For an ATmega32 (a different chip from this lab, running at 8 MHz, so $T_{\text{cyc}} = 125$ ns), a nested-loop `DELAY` subroutine uses $N_1 = 20$ (outer) and $N_2 = 150$ (inner), with a 3-cycle inner loop body (`NOP + DEC + BRNE`). Calculate the total delay by hand, following the same two-level accounting Example 3-18 used, and give your answer in microseconds.

Find the Bug 1: DDRB never set

A student wrote the skeleton’s `MAIN` loop and `DELAY` subroutine correctly, but accidentally deleted the two lines that set `DDRB` before `MAIN`. The program assembles without errors, and `avrdude` reports a successful upload with no verification failure. Predict, qualitatively, what the LED and the oscilloscope trace on D13 will show, and explain *why* – in terms of what a `DDRx` bit versus a `PORTx` bit actually controls on a pin that is still configured as an input – even though every instruction executed exactly as written.

Write Code on Paper 1: A delay subroutine for a different chip

On paper, without a computer, simulator, or AI tool, write a complete `DELAY` subroutine (the two-part structure: set counter(s), then loop) that produces as close as possible to **20 ms** on a chip running at a **4 MHz** crystal. Show every step of your cycle accounting – the number of registers you need, their values, the cycles per pass, and the final delay in microseconds compared to the 20 ms target.

Draw on Paper 1: Predict, then verify, your f_3 trace

Before building Exercise 3's program, sketch on paper the oscilloscope trace you expect: axes labeled (voltage on y , time on x), the two logic levels marked (0 V / 5 V), and the period, half-period, and duty cycle labeled with numbers. After you build, upload, and actually measure it, annotate your sketch with the real measured values next to your predicted ones. Are they close? If your predicted duty cycle wasn't exactly 50%, can you identify which part of the code caused the asymmetry?

Reflection

In one short paragraph, compare this lab's timing model to Lab 1/2's EC-1 machine. On EC-1, every state transition took *exactly* one clock edge – timing was deterministic the moment you designed the FSM, with no measurement needed to know how long anything took. Here, the ATmega328P's crystal sets a real physical time base, but your hand-calculated delay is still only an *approximation* (it ignores setup overhead, and your crystal has some tolerance) – which is why this lab ends with an oscilloscope, not just a calculation. What would you need to add to this program to make its timing as deterministic as EC-1's, without touching the crystal itself? (You are not expected to implement it – Lecture 3's closing note names the AVR feature that does exactly this, covered fully in a later chapter.)

Appendix A: AVRdude Troubleshooting

Message / symptom	Likely cause and fix
<code>stk500_getsync(): attempt N of 10: not in sync</code>	Wrong port, or the bootloader isn't listening. Double-check <code>-P</code> ; press the Uno's reset button just before running <code>avrdude</code> if auto-reset (via DTR) isn't working.
Port open/permission error (e.g. <code>Resource busy</code> , <code>Access is denied</code>)	Another program (Arduino IDE Serial Monitor, a terminal, Microchip Studio's own terminal) is holding the port open. Close it and retry.
Port does not appear in the OS's device list at all	Driver missing for a CH340/CH341 clone USB-serial chip; install the vendor driver. On macOS/Linux, list ports with <code>ls /dev/cu.*</code> or <code>ls /dev/tty*</code> before and after plugging in the board to spot it.
<code>avrdude: verification error, first mismatch at byte 0x0000</code>	Flash didn't actually update – often a stale <code>.hex</code> from a failed rebuild. Rebuild in Studio, confirm the <code>.hex</code> timestamp, and re-upload.
Upload succeeds, but the board does nothing new	Confirm you're pointing <code>avrdude</code> at the <code>.hex</code> you just rebuilt (check the path/timestamp), not an old one from an earlier exercise.